CETAM

Centro de Educação Tecnológica do Amazonas

A P O S T I L A D O A L U N O

Linguagem de Programação

Python

com Visual Studio Code e Google Colaboratory

Projeto integrador: Agência Amazônia Tur Digital

Curso de Qualificação Profissional • 80 horas • Presencial

Professora: Esp. Priscila Gonçalves

E-mail leylapristeacher@gmail.com

GitHb: https://github.com/priscilagon/Logica_Programacao_Basica

Manaus – Amazonas

Sumário

Apresentação 6
O projeto integrador: Agência Amazônia Tur Digital 6
As seis fases do curso 6
Como você será avaliado 7
Como usar esta apostila 7
Legenda dos quadros 7
Fase 1 – Descoberta e Fundamentos 9
Aula 1 – Abertura e conceituação 9
1.1 O que é programar? 9
1.2 Por que Python? 9
1.3 Algoritmos e pseudocódigo 10
1.4 Preparando o ambiente 10
Opção 1 – Google Colab (sem instalação) 10
Opção 2 – Visual Studio Code (no seu computador) 11
Colab ou VS Code? 12
Atividades 12
Aula 2 – Tipos de dados, variáveis, entrada e saída 12
2.1 Saída de dados e comentários 12
2.2 Variáveis 13
2.3 Tipos de dados 13
2.4 Entrada de dados com input() 13
2.5 Conversão de tipos 14
Atividades 14
Aula 3 – Operadores aritméticos 15
3.1 Os operadores 15
3.2 Precedência 15
3.3 Aplicando na agência 15
Atividades 16
Aula 4 – Operadores relacionais e lógicos; estrutura de um algoritmo 16
4.1 Operadores relacionais 16
4.2 Operadores lógicos e tabela-verdade 17
4.3 Estrutura de um algoritmo 17

4.4 Entrega da Fase 1 – Calculadora v1 18
Atividades 18
Fase 2 – Decisão 19
Aula 5 – Estrutura condicional if / else 19
5.1 if e else 19
5.2 Indentação 19
5.3 Aplicando: desconto 19
Atividades 20
Aula 6 – elif, condições encadeadas e compostas 20
6.1 Várias faixas com elif 20
6.2 Condições compostas 21
Atividades 21
Aula 7 – Consolidação da Fase 2 e AV1 21
7.1 Mapa de revisão 22
7.2 Sobre a AV1 22
7.3 Entrega da Fase 2 – Calculadora v2 22
Fase 3 – Repetição 24
Aula 8 – Laço while 24
8.1 Como funciona o while 24
8.2 Acumulador e contador 24
8.3 Validação de entrada 24
8.4 Repetindo o programa inteiro 25
Atividades 25
Aula 9 – Laço for e range() 25
9.1 O laço for 25
9.2 for × while 26
9.3 Gasto dia a dia 26
Atividades 26
Aula 10 – break, continue, pass, menu interativo e AV2 27
10.1 Controlando o laço 27
10.2 O menu do sistema com while True 27
10.3 Sobre a AV2 28
Atividades 28
Fase 4 – Organização de Dados 29
Aula 11 – Listas 29

11.1 Criando e acessando 29
11.2 Métodos e funções úteis 29
11.3 Percorrendo com for 30
Atividades 30
Aula 12 – Tuplas e dicionários 30
12.1 Tuplas 31
12.2 Dicionários 31
12.3 Lista × tupla × dicionário 31
Atividades 32
Aula 13 – Manipulação de strings 32
13.1 Funções de texto 32
13.2 Validações simples 33
13.3 Código de reserva e recibo 33
Atividades 34
Aula 14 – Consolidação da Fase 4 e AV3 34
14.1 Revisão 34
14.2 Cadastro de clientes em memória 34
14.3 Questões de revisão para a AV3 35
Atividades 35
Fase 5 – Modularização e Persistência 36
Aula 15 – Funções com def 36
15.1 Por que modularizar? 36
15.2 Anatomia de uma função 36
Atividades 37
Aula 16 – Parâmetros, argumentos, retorno e escopo 37
16.1 Parâmetros × argumentos 38
16.2 return × print 38
16.3 Valores padrão e retorno de listas 38
16.4 Escopo local e global 38
Checklist de code review (em duplas) 39
Atividades 39
Aula 17 – Arquivos .txt 39
17.1 Por que gravar em arquivos? 39
17.2 Modos de abertura 39
17.3 Escrevendo e lendo 40

17.4 Formato de registro 40
17.5 Onde o arquivo é gravado? 41
Atividades 41
Fase 6 – POO e Integração 43
Aula 18 – Fundamentos de Programação Orientada a Objetos 43
18.1 Classe e objeto 43
18.2 Nossa primeira classe 43
18.3 A classe Usuario 44
Atividades 45
Aula 19 – Desenvolvimento do cofre digital (oficina de projeto) 45
19.1 Requisitos do cofre digital 45
19.2 Trabalho em dupla: pair programming 45
19.3 Estrutura sugerida do programa 45
19.4 Depuração coletiva 47
Aula 20 – Apresentação final, AV4 e encerramento 47
20.1 Roteiro da apresentação 47
20.2 Critérios da AV4 48
20.3 E depois do curso? 48
Anexo A – Erros comuns e depuração 49
Tratando um erro com try/except 49
Técnicas de depuração 49
Anexo B – Resumo de comandos 50
Anexo C – Glossário 51
Anexo D – Gabarito das atividades 52
Aula 1 52
Aula 2 52
Aula 3 52
Aula 4 52
Aula 5 52
Aula 6 53
Aula 8 53
Aula 9 53
Aula 10 54
Aula 11 54
Aula 12 54

Aula 13 54
Aula 14 54
Aula 15 55
Aula 16 55
Aula 17 55
Aula 18 55
Referências 56

Apresentação

Seja bem-vindo(a) ao curso de **Linguagem de Programação Python**! Esta apostila acompanha, aula a aula, as **20 aulas de 4 horas** do curso e foi escrita para quem **nunca programou**. Você vai aprender fazendo: em cada aula há explicações curtas, exemplos para digitar e executar, atividades e um **desafio** que constrói, passo a passo, o projeto integrador do curso.

Para programar, você usará duas ferramentas que se completam: o **Google Colaboratory (Colab)**, que roda no navegador e não exige instalação, e o **Visual Studio Code (VS Code)**, editor profissional instalado no computador. Os exemplos funcionam nas duas; quando houver diferença, ela estará indicada.

O projeto integrador: Agência Amazônia Tur Digital

Ao longo do curso você vai construir o sistema de uma agência de turismo fictícia. Ele reúne as duas práticas profissionais do curso: a **calculadora de despesas de viagem** e o **cofre digital de clientes e reservas**. Os exemplos usam destinos do Amazonas: **Parintins, Novo Airão, Presidente Figueiredo, Maués** e o **Encontro das Águas**. Todos os valores em reais são **fictícios**, criados para fins didáticos.

As seis fases do curso

Fase	Aulas	Foco	Entrega
1. Descoberta e Fundamentos	1–4	Conceitos, variáveis, tipos, operadores, `print()` e `input()`	Calculadora de viagem v1
2. Decisão	5–7	`if`, `else`, `elif`; orçamento de R$ 1.000,00	Calculadora v2
3. Repetição	8–10	`while`, `for`, `break`, `continue`, `pass`	Menu interativo
4. Organização de Dados	11–14	Listas, tuplas, dicionários e strings	Cadastro em memória
5. Modularização e Persistência	15–17	Funções com `def`, parâmetros, retorno, arquivos `.txt`	Sistema modular com dados salvos
6. POO e Integração	18–20	Programação orientada a objetos, cofre digital, apresentação	Cofre digital (sistema final)

Como você será avaliado

Além da avaliação diagnóstica (aula 1) e das avaliações formativas (acompanhamento diário, checklists e devolutivas), o curso tem **quatro atividades avaliativas**. A atribuição de notas é de competência do professor.

Avaliação	Aula	O que será feito
AV1	7	Prática individual em laboratório: calculadora de viagem com decisão.
AV2	10	Prática em duplas: menu com repetição, validação de entrada e controle de fluxo.
AV3	14	Prova prática e teórica individual: listas, tuplas, dicionários e strings.
AV4	20	Projeto final (cofre digital) e apresentação oral em dupla.

Como usar esta apostila

- **Digite os códigos você mesmo.** Copiar e colar não treina a mão nem os olhos para os detalhes da sintaxe.
- **Execute cada exemplo** e observe a saída. Depois altere valores e veja o que muda.
- **Faça as atividades** de cada aula e só então confira o gabarito (Anexo D).
- **Guarde seus arquivos.** O desafio de cada aula usa o código da aula anterior. Salve suas versões (`calculadora_v1.py`, `calculadora_v2.py`...).
- **Errar faz parte.** Mensagens de erro são pistas; o Anexo A ensina a lê-las.

Legenda dos quadros

Nesta aula
Objeto do conhecimento: o conteúdo estudado na aula.
Habilidade a desenvolver: a competência que você vai desenvolver.

Dica
Sugestões para estudar melhor e ganhar tempo.

⚠ Atenção
Pontos que exigem cuidado para não gerar problemas no código.

✕ Erro comum

Erros típicos de quem está começando e como evitá-los.

Python na Amazônia

Exemplos ligados ao nosso contexto regional.

Desafio do projeto

Etapa do projeto integrador

Tarefa que você entrega ao final da aula.

Fase 1 – Descoberta e Fundamentos

Nas quatro primeiras aulas você vai entender o que é programar, preparar seu ambiente e aprender os blocos básicos de qualquer programa: variáveis, tipos de dados, entrada, saída e operadores. Ao final, entregará a **Calculadora de viagem v1**.

Aula 1 – Abertura e conceituação

> **Nesta aula**
> **Objeto do conhecimento:** introdução às linguagens de programação, Python e pseudocódigos.
> **Habilidade a desenvolver:** aplicar os conceitos e fundamentos da linguagem de programação, com clareza e entendimento.
> **Avaliação:** diagnóstica (sem nota) e participação.

1.1 O que é programar?

Um **programa** é uma sequência de instruções que o computador executa na ordem em que foram escritas. **Programar** é escrever essas instruções, resolvendo um problema passo a passo. Fazemos isso em uma **linguagem de programação**, que funciona como um idioma entendido pelo computador.

As linguagens de **alto nível**, como o Python, usam palavras e símbolos próximos da linguagem humana, o que facilita a leitura e a escrita. O programa é então traduzido para a linguagem da máquina por um **interpretador**.

1.2 Por que Python?

Python foi criado por Guido van Rossum e lançado em 1991. É conhecida pela sintaxe clara, o que a torna excelente para iniciantes, e é poderosa o suficiente para uso profissional.

Área	Exemplos de uso do Python
Ciência de dados	Análise de tabelas, estatística e gráficos (pandas, matplotlib).
Inteligência Artificial	Aprendizado de máquina e redes neurais.
Automação	Renomear arquivos, enviar e-mails, preencher planilhas.
Desenvolvimento web	Sites e APIs (Django, Flask, FastAPI).
Pesquisa científica	Processamento de dados de sensores, imagens e simulações.

1.3 Algoritmos e pseudocódigo

Antes de codificar, é útil pensar em um **algoritmo**: um passo a passo para resolver um problema. Quando o escrevemos em português, em linguagem simples, chamamos de **pseudocódigo**. Exemplo: calcular o custo de um passeio de barco.

```
1. Leia o preço do passeio por pessoa
2. Leia a quantidade de pessoas
3. Calcule: custo_total = preço × quantidade
4. Mostre o custo_total na tela
```

Pseudocódigo: não é Python, é só o raciocínio.

> **Dica**
> Se você consegue explicar o passo a passo em português, já fez metade do trabalho. A outra metade é traduzir para Python.

1.4 Preparando o ambiente

Você pode programar em Python de várias formas. Neste curso trabalharemos com o **Colab** e o **VS Code**. Também existe o **IDLE**, editor simples que acompanha a instalação do Python e pode ser usado como alternativa.

Opção 1 – Google Colab (sem instalação)

O **Google Colaboratory** é um serviço gratuito do Google que executa Python no navegador, usando documentos chamados **notebooks** (arquivos `.ipynb`). Basta ter uma conta Google.

1. Acesse **colab.research.google.com** e entre com sua conta Google.
2. Clique em **Arquivo → Novo notebook**.
3. Renomeie o notebook clicando no título, por exemplo, `Aula01_SeuNome`.
4. Na primeira **célula de código**, digite o programa abaixo.
5. Execute com o botão ▶ ou pressionando **Shift + Enter**.

```
print("Olá, Amazonas!")
```

```
Olá, Amazonas!
```

Atalho / elemento	Função
Shift + Enter	Executa a célula e passa para a próxima.
Ctrl + Enter	Executa a célula e permanece nela.
+ Código / + Texto	Adiciona células de código ou de anotações.
Ícone de pasta (lateral)	Painel de arquivos: enviar e baixar arquivos do notebook.

> ⚠ **Atenção**
> As **variáveis ficam na memória** do ambiente. Se o Colab desconectar por inatividade, execute as células novamente, de cima para baixo.
> O notebook é salvo no seu **Google Drive**. Para guardar uma cópia, use **Arquivo → Fazer download**.

Opção 2 – Visual Studio Code (no seu computador)

O **VS Code** é um editor de código gratuito da Microsoft. Para usá-lo com Python, são necessários o **interpretador Python** e a **extensão Python** do VS Code.

1. Acesse **python.org**, baixe a versão estável mais recente e execute o instalador. **No Windows, marque "Add python.exe to PATH"** antes de instalar.
2. No terminal, confira a instalação com o comando abaixo.
3. Baixe e instale o VS Code em **code.visualstudio.com**.
4. No VS Code, abra **Extensões** (Ctrl + Shift + X), pesquise **Python** e instale a extensão publicada pela **Microsoft**.
5. Crie uma pasta (por exemplo, `agencia_tur`) e abra-a em **Arquivo → Abrir Pasta**.
6. Crie o arquivo `ola.py`, digite o código, salve (Ctrl + S) e execute com o botão ▶ (Run Python File) ou pelo terminal.

```
python --version
```

No Linux/macOS pode ser python3 --version. No Windows também funciona: py --version

```
python ola.py
```

Comando para executar o arquivo pelo terminal integrado (menu Terminal → Novo Terminal).

> ✕ **Erro comum**
> "python não é reconhecido como comando": o Python não foi adicionado ao PATH. Reinstale marcando a opção correspondente.
> Salvar o arquivo sem a extensão `.py` impede o VS Code de reconhecê-lo como Python.

Colab ou VS Code?

Critério	Google Colab	VS Code
Instalação	Nenhuma; roda no navegador.	Exige instalar Python e o editor.
Internet	Necessária.	Não é necessária para executar.
Formato	Notebook `.ipynb` (células).	Arquivo `.py` (script).
Melhor para	Aprender, testar ideias, analisar dados.	Projetos com vários arquivos e uso profissional.
Arquivos gerados	Ficam no ambiente temporário (veja a Aula 17).	Ficam na pasta do projeto, no seu computador.

> **Desafio do projeto**
> **Apresentação do desafio da Fase 1**
> Até a Aula 4 você construirá a **Calculadora de despesas de viagem v1** da Agência Amazônia Tur Digital. Crie agora a pasta `agencia_tur` (VS Code) e um notebook `Agencia_Tur` (Colab) para guardar suas versões.

Atividades

1. Escreva o pseudocódigo para “calcular o custo de um passeio de barco para um grupo”.
2. No Colab, execute `print("Olá, Amazonas!")`. Depois, altere a mensagem para incluir seu nome.
3. No VS Code, crie o arquivo `ola.py` e execute-o pelo botão ▶ e pelo terminal.

Aula 2 – Tipos de dados, variáveis, entrada e saída

> **Nesta aula**
> **Objeto do conhecimento:** tipos de dados, declaração de variáveis, `print()` e `input()`.
> **Habilidade a desenvolver:** utilizar os princípios e funções básicas da linguagem Python.
> **Avaliação:** formativa, pelo checklist da prática.

2.1 Saída de dados e comentários

A função `print()` exibe informações na tela. Textos ficam entre aspas, simples ou duplas. Comentários começam com `#` e são ignorados pelo Python: servem para explicar o código.

```
# Mensagem de boas-vindas da agência
print("Bem-vindo à Agência Amazônia Tur Digital!")
print("Destino:", "Parintins")
```

```
Bem-vindo à Agência Amazônia Tur Digital!
```

```
Destino: Parintins
```

2.2 Variáveis

Uma **variável** é um nome que guarda um valor na memória, como uma caixinha etiquetada. O sinal `=` significa "guarde o valor da direita na variável da esquerda".

```
destino = "Novo Airão"
dias = 4
diaria = 180.50
print(destino)
print(dias)
```

```
Novo Airão
4
```

- Nomes podem ter letras, números e _, mas **não podem começar com número** nem ter espaços, acentos ou símbolos.
- Maiúsculas e minúsculas são diferentes: `dias` e `Dias` são variáveis distintas.
- Não use palavras reservadas, como `if`, `for` e `print`.
- Prefira nomes claros em `snake_case`: `gasto_por_dia`, e não `x`.

2.3 Tipos de dados

Tipo	Nome	Exemplos
`int`	Inteiro	`10`, `-3`, `0`
`float`	Decimal	`3.14`, `180.50` (use ponto, não vírgula)
`str`	Texto	`"Maués"`, `'Parintins'`
`bool`	Lógico	`True` ou `False`

```
print(type(4))
print(type(180.50))
print(type("Maués"))
print(type(True))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

2.4 Entrada de dados com input()

A função `input()` faz uma pergunta ao usuário e guarda a resposta. **Tudo o que `input()` devolve é texto (`str`)**, mesmo que a pessoa digite um número.

```
nome = input("Nome do cliente: ")
destino = input("Destino: ")
dias = int(input("Dias de viagem: "))   # converte para inteiro

print(f"{nome} viajará para {destino} por {dias} dias.")
```

```
Nome do cliente: Marina
Destino: Presidente Figueiredo
Dias de viagem: 3
Marina viajará para Presidente Figueiredo por 3 dias.
```

A linha final usa uma **f-string**: o `f` antes das aspas permite colocar variáveis entre chaves `{ }` dentro do texto.

2.5 Conversão de tipos

Função	O que faz
`int(valor)`	Converte para inteiro.
`float(valor)`	Converte para decimal.
`str(valor)`	Converte para texto.

✕ Erro comum

`dias = input("Dias: ")` e depois `dias + 1` gera **TypeError**: não se soma texto com número. Use `int(input(...))`.

Digitar `180,50` (vírgula) e converter com `float()` gera **ValueError**. Use `180.50`.

Dica

No Colab, o campo de digitação do `input()` aparece abaixo da célula; no VS Code, a digitação é feita no terminal integrado.

Desafio do projeto

Tela de entrada de dados do cliente

Crie um programa que peça **nome do cliente, destino e dias de viagem** e mostre um resumo na tela. Salve como `calculadora_v1.py` (ou na célula do notebook).

Atividades

1. Crie um programa que receba nome, destino e dias de viagem e exiba a frase “Fulano viajará para X por Y dias.”
2. Caça aos erros (em duplas): (a) `dias = input("Dias: ") + 1` (b) `1destino = "Parintins"` (c) `print("Total: " + 150)`.

Aula 3 – Operadores aritméticos

Nesta aula

Objeto do conhecimento: operadores `+` `-` `*` `/` `//` `%` `**` e precedência.

Habilidade a desenvolver: utilizar os princípios e funções básicas da linguagem Python.

Avaliação: formativa, pela entrega da lista de exercícios.

3.1 Os operadores

Operador	Operação	Exemplo	Resultado
`+`	Adição	`7 + 2`	`9`
`-`	Subtração	`7 - 2`	`5`
`*`	Multiplicação	`7 * 2`	`14`
`/`	Divisão (sempre decimal)	`7 / 2`	`3.5`
`//`	Divisão inteira	`7 // 2`	`3`
`%`	Resto da divisão	`7 % 2`	`1`
`**`	Potência	`7 ** 2`	`49`

3.2 Precedência

O Python segue a ordem da matemática: **parênteses**, depois `**`, depois `*` `/` `//` `%` e por último `+` `-`. Use parênteses para deixar a intenção clara.

```
print(2 + 3 * 4)       # 14
print((2 + 3) * 4)     # 20
```

3.3 Aplicando na agência

Calcular o custo da viagem exige somar hospedagem, transporte e alimentação **por dia** e multiplicar pelos dias:

```
hospedagem = 200.00
transporte = 50.00
alimentacao = 80.00
dias = 3

gasto_dia = hospedagem + transporte + alimentacao
total = dias * gasto_dia
print(f"Gasto por dia: R$ {gasto_dia:.2f}")
print(f"Total da viagem: R$ {total:.2f}")
```

```
Gasto por dia: R$ 330.00
Total da viagem: R$ 990.00
```

O trecho `:.2f` mostra o número com duas casas decimais, ideal para valores em reais.

Os operadores `//` e `%` são úteis para trocos e divisões em partes:

```
orcamento = 1000
diaria = 330
print(orcamento // diaria)   # 3 dias completos
print(orcamento % diaria)    # sobram R$ 10
```

Desafio do projeto

Cálculo do custo total da viagem

Peça **dias**, **hospedagem por dia**, **transporte por dia** e **alimentação por dia**. Calcule e mostre o gasto diário e o **total** (dias × gasto por dia).

Atividades

1. Leia três notas e mostre a média.
2. Leia o valor de uma compra e o valor pago e calcule o troco.
3. Com orçamento de R$ 1.000,00 e diária de R$ 330,00, mostre quantos dias inteiros cabem no orçamento e quanto sobra.
4. Sem executar, calcule `2 + 3 * 4 ** 2`.

Aula 4 – Operadores relacionais e lógicos; estrutura de um algoritmo

Nesta aula

Objeto do conhecimento: operadores relacionais e lógicos (`==` `!=` `>` `<` `>=` `<=` `and` `or` `not`); estrutura de um algoritmo.

Habilidade a desenvolver: aplicar os conceitos e fundamentos da linguagem de programação.

Avaliação: formativa; Calculadora v1 com devolutiva qualitativa.

4.1 Operadores relacionais

Comparam dois valores e resultam em `True` ou `False`.

Operador	Significado	Exemplo
`==`	Igual a	`5 == 5` → `True`
`!=`	Diferente de	`5 != 3` → `True`
`> <`	Maior / menor que	`2 > 4` → `False`
`>= <=`	Maior ou igual / menor ou igual	`7 >= 7` → `True`

> ⚠ **Atenção**
> `=` **guarda** um valor; `==` **compara** dois valores. Confundir os dois é um dos erros mais comuns.

4.2 Operadores lógicos e tabela-verdade

Combinam condições. Pense em situações reais: "o cliente **tem orçamento** E **tem vaga** no passeio?".

tem_orcamento	tem_vaga	and (E)	or (OU)	not tem_orcamento (NÃO)	not tem_vaga (NÃO)
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	False
False	False	False	False	True	True

O operador `not` inverte o valor: `not True` é `False`. Teste expressões diretamente no console (no Colab, em uma célula; no VS Code, digite `python` no terminal):

```
tem_orcamento = True
tem_vaga = False
print(tem_orcamento and tem_vaga)
print(tem_orcamento or tem_vaga)
print(not tem_vaga)
print(1 <= 5 <= 30)
```

```
False
True
True
True
```

4.3 Estrutura de um algoritmo

Quase todo programa segue três etapas: **entrada → processamento → saída**. Organizar o código assim, com comentários, torna-o mais fácil de ler.

Etapa	O que acontece	Na calculadora
Entrada	O programa recebe os dados.	`input()` de nome, destino, dias e gastos.
Processamento	Os dados são calculados.	Soma dos gastos e multiplicação pelos dias.
Saída	O resultado é mostrado.	`print()` do resumo e do total.

4.4 Entrega da Fase 1 – Calculadora v1

```
# Agência Amazônia Tur Digital - Calculadora de viagem v1

# ----- ENTRADA -----
print("=== Agência Amazônia Tur Digital ===")
nome = input("Nome do cliente: ")
destino = input("Destino: ")
dias = int(input("Quantidade de dias: "))
hospedagem = float(input("Hospedagem por dia (R$): "))
transporte = float(input("Transporte por dia (R$): "))
alimentacao = float(input("Alimentação por dia (R$): "))

# ----- PROCESSAMENTO -----
gasto_dia = hospedagem + transporte + alimentacao
total = dias * gasto_dia

# ----- SAÍDA -----
print()
print(f"Cliente: {nome}")
print(f"Destino: {destino}")
print(f"Gasto por dia: R$ {gasto_dia:.2f}")
print(f"Total da viagem ({dias} dias): R$ {total:.2f}")
```

```
=== Agência Amazônia Tur Digital ===
Nome do cliente: Marina
Destino: Parintins
Quantidade de dias: 3
Hospedagem por dia (R$): 200
Transporte por dia (R$): 50
Alimentação por dia (R$): 80

Cliente: Marina
Destino: Parintins
Gasto por dia: R$ 330.00
Total da viagem (3 dias): R$ 990.00
```

Entrega da fase

Calculadora de viagem v1

Em dupla, entregue a versão funcionando e faça a **revisão cruzada** do código da outra dupla: o programa roda? Os nomes são claros? Há comentários? Os tipos foram convertidos corretamente?

Atividades

1. Sendo `tem_orcamento = True` e `tem_vaga = False`, qual o resultado de `tem_orcamento and tem_vaga`, `tem_orcamento or tem_vaga` e `not tem_vaga`?
2. Escreva uma expressão que seja verdadeira quando `dias` estiver entre 1 e 30.

Fase 2 – Decisão

Até agora o programa executa sempre as mesmas linhas. Nesta fase ele aprenderá a **tomar decisões**: informar se a viagem cabe no orçamento e classificar o pacote por faixa de preço.

Aula 5 – Estrutura condicional if / else

> **Nesta aula**
> **Objeto do conhecimento:** estruturas condicionais: `if`, `else`.
> **Habilidade a desenvolver:** explanar as estruturas condicionais e controle de fluxo em Python, demonstrando confiança.
> **Avaliação:** formativa.

5.1 if e else

A estrutura `if` (se) executa um bloco **somente se** a condição for verdadeira; o `else` (senão) executa quando ela for falsa.

```
idade = int(input("Idade: "))

if idade >= 18:
    print("Maior de idade.")
else:
    print("Menor de idade.")
```

5.2 Indentação

Em Python, o **recuo** (indentação) define o que pertence a cada bloco. Depois dos dois pontos `:`, as linhas seguintes devem ter o mesmo recuo (convenção: **4 espaços**). Não há chaves nem "fim": a organização visual é a própria sintaxe.

> **✕ Erro comum**
> Esquecer os dois pontos `:` gera **SyntaxError**.
> Recuos inconsistentes geram **IndentationError**. No VS Code, a tecla Tab insere o recuo correto.

5.3 Aplicando: desconto

```
idade = int(input("Idade: "))
estudante = input("É estudante? (s/n): ")
valor = 150.00

if idade >= 60 or estudante == "s":
    valor = valor * 0.5          # meia entrada
    print("Meia entrada aplicada.")
else:
```

```
    print("Valor integral.")

print(f"Valor do passeio: R$ {valor:.2f}")
```

> **Desafio do projeto**
> **Dentro ou fora do orçamento?**
> Na calculadora, depois de calcular o total, informe se a viagem está **dentro ou fora do orçamento de R$ 1.000,00**. Se estiver fora, mostre quanto excede.

```
LIMITE = 1000.00

if total <= LIMITE:
    print("Viagem dentro do orçamento de R$ 1000,00.")
else:
    excesso = total - LIMITE
    print(f"Viagem fora do orçamento! Excede em R$ {excesso:.2f}.")
```

Nomes em MAIÚSCULAS indicam, por convenção, valores fixos (constantes).

Atividades

1. Leia a idade e informe se a pessoa é maior de idade.
2. Leia a nota e informe “Aprovado” (nota ≥ 7) ou “Reprovado”.
3. Aplique 20% de desconto (fictício) a clientes com 60 anos ou mais ou estudantes.

Aula 6 – elif, condições encadeadas e compostas

> **Nesta aula**
> **Objeto do conhecimento:** estruturas condicionais: `if`, `else`, `elif`; condições com `and`/`or`.
> **Habilidade a desenvolver:** explanar as estruturas condicionais e controle de fluxo em Python, demonstrando confiança.
> **Avaliação:** formativa.

6.1 Várias faixas com elif

Quando há mais de duas possibilidades, use `elif` (senão se). O Python testa as condições **de cima para baixo** e executa apenas a primeira verdadeira.

Faixa do total	Pacote (valores fictícios)
Até R$ 800,00	Econômico
De R$ 800,01 a R$ 1.500,00	Conforto
Acima de R$ 1.500,00	Premium

```
if total <= 800:
```

```
    pacote = "Econômico"
elif total <= 1500:
    pacote = "Conforto"
else:
    pacote = "Premium"

print(f"Pacote: {pacote}")
```

6.2 Condições compostas

Use `and` e `or` para combinar condições. No exemplo, sugerimos um destino compatível com o valor da viagem:

```
if pacote == "Econômico" and dias <= 3:
    sugestao = "Presidente Figueiredo"
elif pacote == "Conforto":
    sugestao = "Novo Airão"
else:
    sugestao = "Parintins"

print(f"Sugestão de destino: {sugestao}")
```

> **Dica**
> Condições dentro de condições também são possíveis (`if` aninhado), mas muitas vezes `and`/`or` deixam o código mais simples.

> **Desafio do projeto**
> **Classificação por faixa e sugestão de destino**
> A calculadora deve **classificar a viagem** (Econômico, Conforto ou Premium) e **sugerir um destino** compatível com o valor.

Atividades

1. Classifique a média de um aluno: ≥ 7 “Aprovado”, ≥ 5 “Recuperação”, senão “Reprovado”.
2. Um cliente ganha cortesia (um passeio grátis) se `dias >= 5` e `total <= 1500`. Escreva a condição.

Aula 7 – Consolidação da Fase 2 e AV1

> **Nesta aula**
> **Objeto do conhecimento:** conteúdos das aulas 1 a 6.
> **Habilidade a desenvolver:** aplicar os conceitos e fundamentos da linguagem de programação, com clareza e entendimento.
> **Avaliação:** AV1 – prática individual em laboratório.

7.1 Mapa de revisão

Tema	Pontos para revisar
Fundamentos	`print()`, `input()`, variáveis, tipos, conversão de tipos.
Operadores	Aritméticos, relacionais e lógicos; precedência.
Decisão	`if`, `else`, `elif`, condições compostas, indentação.
Boas práticas	Nomes claros, comentários, entrada → processamento → saída.

7.2 Sobre a AV1

A AV1 é uma **prática individual em laboratório**: você desenvolverá o programa da agência com **entrada de nome, dias e gasto por dia, cálculo do total e decisão sobre o orçamento, com `elif` para a faixa do pacote**. Os critérios são:

- **funcionamento:** o programa roda e produz os resultados corretos;
- **lógica:** as decisões estão corretas;
- **legibilidade:** nomes claros, indentação e comentários;
- **tratamento correto dos tipos:** conversões com `int()` e `float()`.

7.3 Entrega da Fase 2 – Calculadora v2

```
# Agência Amazônia Tur Digital - Calculadora de viagem v2
LIMITE = 1000.00

nome = input("Nome do cliente: ")
dias = int(input("Quantidade de dias: "))
gasto_dia = float(input("Gasto por dia (R$): "))

total = dias * gasto_dia

if total <= 800:
    pacote = "Econômico"
elif total <= 1500:
    pacote = "Conforto"
else:
    pacote = "Premium"

print(f"\nCliente: {nome}")
print(f"Total: R$ {total:.2f} - Pacote {pacote}")

if total <= LIMITE:
    print("Dentro do orçamento de R$ 1000,00.")
else:
    print(f"Fora do orçamento! Excede em R$ {total - LIMITE:.2f}.")
```

Entrega da fase

Calculadora de viagem v2

Programa com decisão sobre o orçamento e classificação por faixa de pacote.

Dica

Durante a prova, execute o programa várias vezes com valores diferentes (um abaixo, um acima e um exatamente igual ao limite). Isso revela erros de lógica.

Fase 3 – Repetição

Nesta fase o programa aprende a **repetir tarefas**: validar entradas, calcular várias viagens em sequência, listar os gastos dia a dia e, por fim, funcionar como um **menu interativo**.

Aula 8 – Laço while

> **Nesta aula**
> **Objeto do conhecimento:** estruturas de repetição: `while`.
> **Habilidade a desenvolver:** usar as estruturas/laços de repetição com domínio.
> **Avaliação:** formativa.

8.1 Como funciona o while

O `while` repete um bloco **enquanto** a condição for verdadeira. Ele precisa de uma **condição de parada**: algo que, em algum momento, torne a condição falsa.

```
contador = 1
while contador <= 3:
    print(f"Passageiro {contador}")
    contador = contador + 1     # atualiza o contador
print("Embarque concluído!")
```

```
Passageiro 1
Passageiro 2
Passageiro 3
Embarque concluído!
```

> **✕ Erro comum**
> Se a condição nunca ficar falsa (por exemplo, esquecer de atualizar o contador), ocorre um **laço infinito**. No VS Code, interrompa com Ctrl + C no terminal; no Colab, clique no botão ■ da célula.

8.2 Acumulador e contador

```
soma = 0
gasto = float(input("Gasto (0 para encerrar): "))
while gasto != 0:
    soma += gasto               # acumulador: soma = soma + gasto
    gasto = float(input("Gasto (0 para encerrar): "))
print(f"Total dos gastos: R$ {soma:.2f}")
```

8.3 Validação de entrada

Podemos repetir a pergunta **até o usuário digitar um valor válido**:

```
dias = int(input("Dias de viagem (1 a 30): "))
while dias < 1 or dias > 30:
```

```
    print("Valor inválido! Digite de 1 a 30.")
    dias = int(input("Dias de viagem (1 a 30): "))
```

8.4 Repetindo o programa inteiro

```
continuar = "s"
while continuar == "s":
    dias = int(input("Dias: "))
    gasto_dia = float(input("Gasto por dia: "))
    print(f"Total: R$ {dias * gasto_dia:.2f}")
    continuar = input("Deseja continuar? (s/n): ").lower()
print("Obrigado por usar a calculadora!")
```

Desafio do projeto

Várias viagens em sequência

Faça a calculadora **repetir** o cálculo e, ao final de cada viagem, perguntar **"Deseja continuar? (s/n)"**. Inclua também a validação dos dias (1 a 30).

Atividades

1. Peça uma nota entre 0 e 10 e repita a pergunta até o valor ser válido.
2. Some gastos digitados pelo usuário até que ele digite 0.

Aula 9 – Laço for e range()

Nesta aula

Objeto do conhecimento: estruturas de repetição: `for`, `range`.

Habilidade a desenvolver: usar as estruturas/laços de repetição com domínio.

Avaliação: formativa.

9.1 O laço for

O `for` é ideal quando sabemos **quantas vezes** repetir. A função `range(início, fim, passo)` gera uma sequência de números; o **fim não é incluído**.

Forma	Valores gerados
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8
`range(5, 0, -1)`	5, 4, 3, 2, 1 (contagem regressiva)

```
for i in range(5, 0, -1):
    print(i)
```

```
print("Embarcar!")
```

```
5
4
3
2
1
Embarcar!
```

9.2 for × while

Use `for` quando o número de repetições é conhecido; use `while` quando a repetição depende de uma condição (como o usuário digitar “n”). O mesmo laço de 1 a 3 escrito das duas formas:

```
for i in range(1, 4):
    print(i)

i = 1
while i <= 3:
    print(i)
    i += 1
```

9.3 Gasto dia a dia

```
dias = 3
gasto_dia = 330.00
acumulado = 0

for dia in range(1, dias + 1):
    acumulado += gasto_dia
    print(f"Dia {dia}: R$ {gasto_dia:.2f} | Acumulado: R$ {acumulado:.2f}")
```

```
Dia 1: R$ 330.00 | Acumulado: R$ 330.00
Dia 2: R$ 330.00 | Acumulado: R$ 660.00
Dia 3: R$ 330.00 | Acumulado: R$ 990.00
```

Desafio do projeto

Extrato da viagem

Exiba o **gasto de cada dia** (Dia 1, Dia 2...) e o **valor acumulado** ao final de cada dia.

Atividades

1. Mostre a tabuada de um número lido (de 1 a 10).
2. Some as diárias de 1 a `dias`, sendo que a diária do dia 1 é R$ 100 e aumenta R$ 20 por dia.
3. Faça uma contagem regressiva de 10 até 1 e, no final, mostre “Embarcar!”.

Aula 10 – break, continue, pass, menu interativo e AV2

Nesta aula

Objeto do conhecimento: comandos de controle de fluxo: `break`, `continue`, `pass`.

Habilidade a desenvolver: utilizar comandos de controle de fluxo.

Avaliação: AV2 – prática em duplas, com registro de colaboração.

10.1 Controlando o laço

- `break` **interrompe** o laço imediatamente.
- `continue` **pula** para a próxima repetição, ignorando o restante do bloco.
- `pass` **não faz nada**: serve para deixar um bloco "reservado" para ser escrito depois.

```
for numero in range(1, 11):
    if numero == 4:
        continue        # pula o 4
    if numero == 8:
        break           # encerra no 8
    print(numero)
```

```
1
2
3
5
6
7
```

```
if opcao == "3":
    pass    # será implementado depois
```

10.2 O menu do sistema com while True

`while True` cria um laço que só termina com `break`. É a base de qualquer menu interativo:

```
destinos = ["Parintins", "Novo Airão", "Presidente Figueiredo", "Maués"]

while True:
    print("\n=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===")
    print("1 - Calcular viagem")
    print("2 - Ver destinos")
    print("0 - Sair")
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        dias = int(input("Dias: "))
        gasto_dia = float(input("Gasto por dia: "))
        print(f"Total: R$ {dias * gasto_dia:.2f}")
    elif opcao == "2":
        for destino in destinos:
            print("-", destino)
    elif opcao == "0":
        print("Até logo!")
```

```
        break
    else:
        print("Opção inválida!")
        continue
```

```
=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
0 - Sair
Escolha uma opção: 2
- Parintins
- Novo Airão
- Presidente Figueiredo
- Maués
```

10.3 Sobre a AV2

A AV2 é uma **prática em duplas**: o menu completo, com **repetição, validação de entrada e uso adequado de `break` e `continue`**. Cada integrante registra sua contribuição.

Critério	O que observar
Funcionamento	O menu repete até o usuário escolher sair; as opções executam corretamente.
Validação	Entradas inválidas são tratadas sem derrubar o programa.
Controle de fluxo	`break` e `continue` usados no lugar certo.
Colaboração	Divisão de tarefas, comunicação e registro de cada contribuição.

Entrega da fase

Menu interativo do sistema

Menu com **1 – Calcular viagem**, **2 – Ver destinos** e **0 – Sair**, integrando a calculadora v2 e a validação de entradas.

Atividades

1. Em um `for` de 1 a 5, se o número for 3 use `continue`; se for 5 use `break`; nos demais casos, mostre o número. Qual é a saída?
2. Acrescente ao menu a opção “3 – Ver orçamento limite” que mostre o valor de R$ 1.000,00.

Fase 4 – Organização de Dados

Até aqui cada variável guardava um único valor. Agora você vai aprender a guardar e organizar **muitos dados** de uma vez: listas, tuplas, dicionários e textos (strings). Ao final, o sistema terá um **cadastro de clientes e destinos em memória**.

Aula 11 – Listas

> **Nesta aula**
> **Objeto do conhecimento:** estruturas de dados em Python: listas; métodos e funções.
> **Habilidade a desenvolver:** aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.
> **Avaliação:** formativa.

11.1 Criando e acessando

Uma **lista** é uma coleção ordenada e **modificável**, escrita entre colchetes. Cada item tem uma posição (índice) que **começa em 0**.

```
destinos = ["Parintins", "Novo Airão", "Presidente Figueiredo", "Maués"]

print(destinos[0])      # primeiro item
print(destinos[-1])     # último item
print(destinos[1:3])    # fatia: posições 1 e 2
print(len(destinos))    # quantidade de itens
```

```
Parintins
Maués
['Novo Airão', 'Presidente Figueiredo']
4
```

11.2 Métodos e funções úteis

Comando	O que faz
`lista.append(x)`	Adiciona `x` ao final.
`lista.insert(i, x)`	Insere `x` na posição `i`.
`lista.remove(x)`	Remove a primeira ocorrência de `x`.
`lista.pop()`	Remove e devolve o último item.
`lista.sort()`	Ordena a lista.
`x in lista`	Verifica se `x` está na lista.
`len()`, `sum()`, `min()`, `max()`	Quantidade, soma, menor e maior valores.

11.3 Percorrendo com for

```
gastos = [990.00, 1250.50, 780.00]

for gasto in gastos:
    print(f"R$ {gasto:.2f}")

print(f"Maior: {max(gastos)} | Menor: {min(gastos)}")
print(f"Média: {sum(gastos) / len(gastos):.2f}")
```

```
R$ 990.00
R$ 1250.50
R$ 780.00
Maior: 1250.5 | Menor: 780.0
Média: 1006.83
```

> **Desafio do projeto**
> **Cadastro de destinos e média de gastos**
> Permita **cadastrar vários destinos** e o gasto de cada um em duas listas e, ao final, calcule a **média dos gastos**.

```
destinos = []
gastos = []

continuar = "s"
while continuar == "s":
    destinos.append(input("Destino: "))
    gastos.append(float(input("Gasto total (R$): ")))
    continuar = input("Cadastrar outro? (s/n): ").lower()

print(f"Média dos gastos: R$ {sum(gastos) / len(gastos):.2f}")
```

Atividades

1. Crie uma lista com 4 destinos. Mostre o primeiro e o último, adicione um quinto e ordene a lista.
2. Dada a lista de gastos `[990, 1250.5, 780]`, mostre o maior, o menor e a média.

Aula 12 – Tuplas e dicionários

> **Nesta aula**
> **Objeto do conhecimento:** estruturas de dados em Python: tuplas e dicionários.
> **Habilidade a desenvolver:** aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.
> **Avaliação:** formativa.

12.1 Tuplas

A **tupla** é parecida com a lista, porém **imutável**: depois de criada, não pode ser alterada. Usa parênteses e serve para dados fixos.

```
coordenadas_manaus = (-3.119, -60.021)
print(coordenadas_manaus[0])
```

12.2 Dicionários

O **dicionário** guarda pares **chave: valor**, entre chaves `{ }`. Em vez de posição, acessamos o dado pela chave, como em uma ficha cadastral.

```
destino = {
    "nome": "Parintins",
    "preco": 950.00,
    "dias": 3
}

print(destino["nome"])
destino["preco"] = 990.00          # altera um valor
destino["transporte"] = "barco"    # adiciona nova chave

for chave, valor in destino.items():
    print(f"{chave}: {valor}")
```

```
Parintins
nome: Parintins
preco: 990.0
dias: 3
transporte: barco
```

12.3 Lista × tupla × dicionário

	Lista	Tupla	Dicionário
Símbolo	`[ ]`	`( )`	`{ chave: valor }`
Mutável?	Sim	Não	Sim
Acesso por	Posição	Posição	Chave
Uso típico	Conjunto de itens	Dados fixos	Ficha com campos

> **Desafio do projeto**
> **Catálogo de pacotes da agência**
> Monte o **catálogo de pacotes** em um dicionário, em que cada destino tem preço e duração, e exiba-o com `.items()`.

```
catalogo = {
    "Parintins": {"preco": 950.00, "dias": 3},
    "Novo Airão": {"preco": 1200.00, "dias": 4},
    "Presidente Figueiredo": {"preco": 780.00, "dias": 2},
```

```
    "Maués": {"preco": 1350.00, "dias": 4},
}

for nome, dados in catalogo.items():
    print(f"{nome}: R$ {dados['preco']:.2f} em {dados['dias']} dias")
```

Preços fictícios, apenas para o exercício.

```
Parintins: R$ 950.00 em 3 dias
Novo Airão: R$ 1200.00 em 4 dias
Presidente Figueiredo: R$ 780.00 em 2 dias
Maués: R$ 1350.00 em 4 dias
```

Atividades

1. Crie um dicionário com nome, preço e duração de um passeio e percorra-o com `.items()`.
2. Explique a diferença entre uma lista e uma tupla e dê um exemplo de uso de cada uma.

Aula 13 – Manipulação de strings

Nesta aula

Objeto do conhecimento: funções para manipulação de strings.

Habilidade a desenvolver: utilizar funções de forma eficaz para manipular conjuntos de caracteres e palavras.

Avaliação: formativa.

13.1 Funções de texto

Comando	O que faz
`texto.upper()` / `.lower()`	Converte para MAIÚSCULAS / minúsculas.
`texto.title()`	Primeira letra de cada palavra em maiúscula.
`texto.strip()`	Remove espaços do início e do fim.
`texto.split(sep)`	Divide o texto em uma lista.
`texto.replace(a, b)`	Troca `a` por `b`.
`texto.find(x)`	Posição de `x` (ou -1 se não achar).
`len(texto)`	Quantidade de caracteres.

```
nome = "   maria da SILVA  "
print(nome.strip().title())          # Maria Da Silva
print("Manaus-AM".split("-"))        # ['Manaus', 'AM']
print("Novo Airão".replace(" ", "_"))
print("Parintins"[0:3])              # Par
```

```
Maria Da Silva
```

```
['Manaus', 'AM']
Novo_Airão
Par
```

13.2 Validações simples

```
email = input("E-mail: ").strip().lower()
if "@" in email and "." in email:
    print("E-mail com formato aceitável.")
else:
    print("E-mail inválido.")

cpf = input("CPF fictício (só números): ")
if len(cpf) == 11 and cpf.isdigit():
    print("CPF com formato válido.")
else:
    print("CPF deve ter 11 dígitos numéricos.")
```

> ⚠ **Atenção**
> Estas validações verificam apenas o **formato**. Nunca digite documentos reais em exercícios: use sempre números fictícios.

13.3 Código de reserva e recibo

```
nome = "marina costa"
destino = "Parintins"
dias = 3
total = 990.00

codigo = f"{destino[:3].upper()}-{dias:02d}-{nome[:2].upper()}"

print("=" * 34)
print("     RECIBO - AMAZÔNIA TUR DIGITAL")
print("=" * 34)
print(f"Cliente: {nome.title()}")
print(f"Destino: {destino}")
print(f"Duração: {dias} dias")
print(f"Total:   R$ {total:.2f}")
print(f"Reserva: {codigo}")
```

```
==================================
     RECIBO - AMAZÔNIA TUR DIGITAL
==================================
Cliente: Marina Costa
Destino: Parintins
Duração: 3 dias
Total:   R$ 990.00
Reserva: PAR-03-MA
```

Em `{dias:02d}`, o `02d` mostra o número inteiro com dois dígitos (03, 12...).

> **Desafio do projeto**
> **Cadastro padronizado e recibo**
> Padronize o **nome do cliente** (sem espaços sobrando, com iniciais maiúsculas) e monte o **recibo da viagem** com f-strings.

Atividades

1. Padronize o texto `"  joão PEDRO  "` para `"João Pedro"`.
2. Valide um e-mail digitado (deve conter `@` e `.`).
3. Gere o código de reserva no formato `PAR-03-MA` a partir do destino, dos dias e do nome do cliente.

Aula 14 – Consolidação da Fase 4 e AV3

> **Nesta aula**
> **Objeto do conhecimento:** listas, tuplas, dicionários, métodos e funções, strings.
> **Habilidade a desenvolver:** aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.
> **Avaliação:** AV3 – prova prática e teórica individual.

14.1 Revisão

- **Listas:** índices, fatiamento, `append`, `remove`, `sort`, `len`, `sum`, `max`, `min`.
- **Tuplas:** imutáveis; dados fixos.
- **Dicionários:** chave-valor, `.items()`, alteração e inclusão de chaves.
- **Strings:** `upper`, `lower`, `strip`, `split`, `replace`, `find`, fatiamento e f-strings.

14.2 Cadastro de clientes em memória

Uma estrutura muito comum é uma **lista de dicionários**: cada dicionário é um cliente e a lista reúne todos.

```
clientes = []

cliente = {
    "nome": input("Nome: ").strip().title(),
    "destino": input("Destino: ").strip().title(),
    "dias": int(input("Dias: "))
}
clientes.append(cliente)

for cl in clientes:
    print(f"{cl['nome']} -> {cl['destino']} ({cl['dias']} dias)")
```

Entrega da fase

Cadastro de clientes e destinos em memória

O menu agora cadastra clientes (lista de dicionários) e mostra o catálogo de pacotes. Os dados ainda se perdem ao fechar o programa: na Fase 5 vamos resolver isso.

14.3 Questões de revisão para a AV3

Atividades

1. Dada `nomes = ["Ana", "Bia", "Caio"]`, o que `nomes[-1]` e `nomes[0:2]` mostram?
2. Qual a diferença entre `lista.append(x)` e `lista.remove(x)`?
3. Dado `aluno = {"nome": "Ana", "nota": 9}`, como alterar a nota para 10? E como acrescentar a chave `"curso"`?
4. O que `"  Maués ".strip().upper()` devolve?

Na AV3 você também resolverá, em laboratório, um problema como o **cadastro de clientes com dicionário e lista**.

Fase 5 – Modularização e Persistência

O sistema já faz muita coisa, mas o código está ficando longo. Nesta fase você vai **dividir o programa em funções** reutilizáveis e **gravar os dados em arquivos de texto**, para que não se percam quando o programa terminar.

Aula 15 – Funções com def

> **Nesta aula**
> **Objeto do conhecimento:** funções em Python: criação e chamada de funções; definindo funções com `def`.
> **Habilidade a desenvolver:** aplicar o recurso de modularização e métodos para o uso de funções e parâmetros.
> **Avaliação:** formativa.

15.1 Por que modularizar?

Uma **função** é um bloco de código com nome que pode ser usado várias vezes. Funções evitam repetição, deixam o programa organizado e facilitam encontrar erros. Você já usou funções prontas, como `print()` e `len()`; agora vai criar as suas.

15.2 Anatomia de uma função

```
def exibir_boas_vindas():
    print("=== Agência Amazônia Tur Digital ===")
    print("Bem-vindo(a)!")

exibir_boas_vindas()     # chamada da função
exibir_boas_vindas()     # reaproveitamento
```

- `def` inicia a definição; depois vêm o **nome**, os parênteses e os dois pontos `:`.
- O corpo da função fica **indentado**.
- A função só é executada quando **chamada**, escrevendo seu nome com parênteses.
- Defina a função **antes** de chamá-la.

> **Desafio do projeto**
> **Refatorar o sistema**
> Reorganize o sistema em funções: `calcular_total()`, `exibir_menu()` e `cadastrar_cliente()`.

```
def exibir_menu():
    print("\n=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===")
    print("1 - Calcular viagem")
```

```
    print("2 - Cadastrar cliente")
    print("3 - Listar clientes")
    print("0 - Sair")
    return input("Escolha: ")

def calcular_total(dias, gasto_dia):
    return dias * gasto_dia

def cadastrar_cliente(clientes):
    nome = input("Nome: ").strip().title()
    destino = input("Destino: ").strip().title()
    dias = int(input("Dias: "))
    clientes.append({"nome": nome, "destino": destino, "dias": dias})
    print("Cliente cadastrado!")

clientes = []
while True:
    opcao = exibir_menu()
    if opcao == "1":
        dias = int(input("Dias: "))
        gasto = float(input("Gasto por dia: "))
        print(f"Total: R$ {calcular_total(dias, gasto):.2f}")
    elif opcao == "2":
        cadastrar_cliente(clientes)
    elif opcao == "3":
        for cl in clientes:
            print(f"{cl['nome']} -> {cl['destino']} ({cl['dias']} dias)")
    elif opcao == "0":
        break
    else:
        print("Opção inválida!")
```

Atividades

1. Crie a função `saudacao_cliente(nome)` que mostre “Olá, [nome]! Boa viagem!”.
2. Transforme o cálculo do total (dias × gasto por dia) em uma função e chame-a duas vezes com valores diferentes.

Aula 16 – Parâmetros, argumentos, retorno e escopo

Nesta aula

Objeto do conhecimento: passagem de parâmetros e retorno; parâmetros, argumentos e retorno.

Habilidade a desenvolver: aplicar o recurso de modularização e métodos para o uso de funções e parâmetros.

Avaliação: formativa, com checklist de code review.

16.1 Parâmetros × argumentos

Parâmetros são as variáveis na definição da função; **argumentos** são os valores enviados na chamada.

```
def calcular_total(dias, gasto_dia):     # dias e gasto_dia: parâmetros
    return dias * gasto_dia

total = calcular_total(3, 330.00)        # 3 e 330.00: argumentos
print(total)
```

```
990.0
```

16.2 return × print

> ⚠ **Atenção**
> `print()` **mostra** algo na tela; `return` **devolve** um valor para ser usado pelo programa (guardado em variável, comparado etc.). Uma função sem `return` devolve `None`.

16.3 Valores padrão e retorno de listas

```
def validar_orcamento(total, limite=1000.00):
    return total <= limite

print(validar_orcamento(990))          # True
print(validar_orcamento(1200))         # False
print(validar_orcamento(1200, 1500))   # True

def listar_nomes(clientes):
    return [cl["nome"] for cl in clientes]
```

16.4 Escopo local e global

Variáveis criadas **dentro** da função são **locais**: só existem enquanto ela executa. Prefira enviar informações por **parâmetros** e recebê-las por **retorno**, em vez de depender de variáveis globais.

```
def exemplo():
    mensagem = "variável local"
    print(mensagem)

exemplo()
# print(mensagem)   # NameError: mensagem não existe fora da função
```

> 🚀 **Desafio do projeto**
> **Busca de destino e validação do orçamento**
> Crie `buscar_destino(nome)`, que procura o destino no catálogo e devolve seus dados (ou `None` se não existir), e `validar_orcamento(total, limite)`. Integre-as ao menu.

```
def buscar_destino(nome, catalogo):
    return catalogo.get(nome.strip().title())    # devolve None se não achar
```

```
dados = buscar_destino("parintins", catalogo)
if dados is None:
    print("Destino não encontrado.")
else:
    print(f"Preço: R$ {dados['preco']:.2f}")
```

Checklist de code review (em duplas)

- Cada função faz **uma coisa** e tem nome que explica o que faz?
- Os parâmetros e o retorno estão corretos? Usa `return` onde precisa devolver valor?
- Há código repetido que poderia virar função?
- O programa funciona com entradas inválidas?

Atividades

1. Crie `eh_par(n)` que devolva `True` ou `False`.
2. Crie `buscar_destino(nome, catalogo)` e teste com um nome existente e outro inexistente.
3. Explique com suas palavras a diferença entre `print` e `return`.

Aula 17 – Arquivos .txt

> **Nesta aula**
> **Objeto do conhecimento:** uso de arquivos `.txt`: leitura e escrita, modos de arquivos e formatos.
> **Habilidade a desenvolver:** abranger a escrita de dados em arquivos, com satisfação.
> **Avaliação:** formativa.

17.1 Por que gravar em arquivos?

Dados guardados em variáveis, listas e dicionários **desaparecem** quando o programa termina. Para guardá-los, usamos **arquivos**. Aqui trabalharemos com arquivos de texto (`.txt`).

17.2 Modos de abertura

Modo	Nome	O que faz
`"r"`	Leitura	Lê o arquivo. Dá erro se ele não existir.
`"w"`	Escrita	Cria o arquivo; **apaga o conteúdo anterior** se já existir.
`"a"`	Acréscimo	Adiciona ao final, mantendo o conteúdo anterior.

17.3 Escrevendo e lendo

A forma recomendada é `with open(...)`, que **fecha o arquivo automaticamente**. Use `encoding="utf-8"` para preservar acentos.

```
destinos = ["Parintins", "Novo Airão", "Maués"]

# Escrita
with open("destinos.txt", "w", encoding="utf-8") as arquivo:
    for destino in destinos:
        arquivo.write(destino + "\n")

# Leitura
with open("destinos.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

```
Parintins
Novo Airão
Maués
```

O `"\n"` (quebra de linha) separa os registros e o `.strip()` remove essa quebra ao ler. Também existem `arquivo.read()` (lê tudo de uma vez) e `arquivo.readline()` (lê uma linha por vez).

17.4 Formato de registro

Para guardar mais de um dado por linha, escolha um separador que não apareça nos textos, como `;`:

```
def salvar_clientes(clientes, arquivo="clientes.txt"):
    with open(arquivo, "w", encoding="utf-8") as f:
        for cl in clientes:
            f.write(f"{cl['nome']};{cl['destino']};{cl['dias']}\n")

def carregar_clientes(arquivo="clientes.txt"):
    clientes = []
    try:
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, destino, dias = linha.strip().split(";")
                clientes.append({"nome": nome, "destino": destino, "dias":
int(dias)})
    except FileNotFoundError:
        pass            # primeira execução: o arquivo ainda não existe
    return clientes
```

try/except evita o erro quando o arquivo ainda não existe. Veja o Anexo A.

17.5 Onde o arquivo é gravado?

	Google Colab	VS Code
Local	Ambiente temporário do notebook (painel de arquivos, ícone de pasta).	Pasta do projeto aberta no VS Code.
Atenção	Os arquivos **somem** quando a sessão é encerrada.	Abra a **pasta** (Arquivo → Abrir Pasta), senão o arquivo pode ir parar em outro lugar.
Guardar	Baixe pelo painel de arquivos ou monte o Google Drive (veja abaixo).	Já fica salvo no computador.

```
# Somente no Colab: salvar em seu Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Depois use caminhos como:
# '/content/drive/MyDrive/agencia_tur/clientes.txt'
```

O Colab pede sua autorização para acessar o Drive.

> **Desafio do projeto**
>
> **Persistência de clientes e reservas**
>
> Faça o sistema **carregar** os dados do arquivo ao iniciar e **salvar** ao cadastrar um cliente, para que as informações permaneçam entre uma execução e outra.

```
clientes = carregar_clientes()      # lê o arquivo ao iniciar

while True:
    opcao = exibir_menu()
    if opcao == "2":
        cadastrar_cliente(clientes)
        salvar_clientes(clientes)   # grava após cada cadastro
    elif opcao == "0":
        break
    # ... demais opções do menu
```

> **Entrega da fase**
>
> **Sistema modular com dados salvos**
>
> Menu com funções, cadastro de clientes e arquivo `clientes.txt` carregado ao iniciar. Combine com sua dupla o plano do **cofre digital** da próxima fase.

Atividades

1. Grave uma lista de 5 destinos em `destinos.txt` e depois leia e mostre cada um.
2. Leia o arquivo `destinos.txt` e mostre quantas linhas ele possui.

3. Qual a diferença entre os modos `"w"` e `"a"`? O que acontece se usarmos `"w"` por engano em um arquivo com dados?

> **⚠ Atenção**
> Se algum dado digitado contiver o separador `;`, a leitura vai falhar. Em sistemas reais, isso é tratado na validação da entrada.

Fase 6 – POO e Integração

Na última fase você conhecerá os fundamentos da **Programação Orientada a Objetos (POO)** e vai juntar tudo o que aprendeu em um sistema completo: o **cofre digital** da agência, com cadastro de usuário, senha, login e dados gravados em arquivo.

Aula 18 – Fundamentos de Programação Orientada a Objetos

> **Nesta aula**
> **Objeto do conhecimento:** introdução à Programação Orientada a Objetos; fundamentos de POO.
> **Habilidade a desenvolver:** executar os conceitos iniciais de programação orientada a objetos (POO), com clareza e compreensão.
> **Avaliação:** formativa.

18.1 Classe e objeto

Na POO organizamos o programa em **objetos**, que reúnem **dados** (atributos) e **ações** (métodos). A **classe** é o molde; o **objeto** é cada exemplar criado a partir dele. Analogia: a classe é o **molde** do pacote de viagem; cada pacote vendido é um **objeto**.

Conceito	Significado
Classe	Molde que define quais atributos e métodos os objetos terão.
Objeto	Exemplar criado a partir da classe (instância).
Atributo	Dado do objeto (ex.: nome, destino).
Método	Função que pertence à classe (ex.: `resumo()`).
__init__	Método especial que prepara o objeto quando ele é criado (construtor).
self	Referência ao próprio objeto; sempre o primeiro parâmetro dos métodos.

18.2 Nossa primeira classe

```
class Cliente:
    def __init__(self, nome, destino):
        self.nome = nome
        self.destino = destino

    def resumo(self):
        return f"{self.nome} viajará para {self.destino}"

cliente1 = Cliente("Marina", "Parintins")
cliente2 = Cliente("João", "Maués")

print(cliente1.resumo())
```

```
print(cliente2.resumo())
```

```
Marina viajará para Parintins
João viajará para Maués
```

```
class Pacote:
    def __init__(self, destino, preco, dias):
        self.destino = destino
        self.preco = preco
        self.dias = dias

    def descricao(self):
        return f"{self.destino}: R$ {self.preco:.2f} em {self.dias} dias"

pacote = Pacote("Novo Airão", 1200.00, 4)
print(pacote.descricao())
```

```
Novo Airão: R$ 1200.00 em 4 dias
```

> ❌ **Erro comum**
> Esquecer o `self` na definição dos métodos ou dentro da classe (`self.nome`, e não só `nome`) é o erro mais comum com classes.

18.3 A classe Usuario

> 🚀 **Desafio do projeto**
> **Classe Usuario**
> Crie a classe `Usuario` (nome, senha) com os métodos `cadastrar()` e `autenticar()`. Os dados serão gravados no arquivo `usuarios.txt`.

```
import os

class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha

    def cadastrar(self, arquivo="usuarios.txt"):
        with open(arquivo, "a", encoding="utf-8") as f:
            f.write(f"{self.nome};{self.senha}\n")

    def autenticar(self, arquivo="usuarios.txt"):
        if not os.path.exists(arquivo):
            return False
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, senha = linha.strip().split(";")
                if nome == self.nome and senha == self.senha:
                    return True
        return False
```

> ⚠ **Atenção**
> Guardar senhas em texto puro é aceitável **somente para estudo**. Sistemas reais nunca fazem isso: as senhas são transformadas por funções de hash e armazenadas de forma protegida.

Atividades

1. Crie a classe `Pacote` com os atributos `destino`, `preco` e `dias` e o método `descricao()`. Crie dois objetos e mostre suas descrições.
2. Crie a classe `Cliente` com um método `dobrar_dias()` que dobre o atributo `dias`.

Aula 19 – Desenvolvimento do cofre digital (oficina de projeto)

> **Nesta aula**
> **Objeto do conhecimento:** integração de funções, condicionais, repetição, arquivos e POO.
> **Habilidade a desenvolver:** desenvolver programas funcionais em Python com raciocínio lógico, criatividade e resiliência, corrigindo possíveis erros com a colaboração em equipe.
> **Avaliação:** formativa, com acompanhamento do docente e registro do progresso.

19.1 Requisitos do cofre digital

- cadastro de usuário com nome e senha;
- login (autenticação) com verificação do arquivo;
- mensagem personalizada após o login;
- todos os dados **persistidos em arquivo**;
- menu com repetição, validação de entradas e código organizado em funções e classe.

19.2 Trabalho em dupla: pair programming

Na programação em par, um aluno é o **piloto** (digita o código) e o outro é o **copiloto** (revisa, sugere e procura erros). **Alternem os papéis** com frequência. Todos aprendem mais e os erros aparecem mais cedo.

19.3 Estrutura sugerida do programa

```
import os

class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha

    def cadastrar(self, arquivo="usuarios.txt"):
        with open(arquivo, "a", encoding="utf-8") as f:
            f.write(f"{self.nome};{self.senha}\n")
```

```
    def autenticar(self, arquivo="usuarios.txt"):
        if not os.path.exists(arquivo):
            return False
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, senha = linha.strip().split(";")
                if nome == self.nome and senha == self.senha:
                    return True
        return False

def exibir_menu():
    print("\n=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===")
    print("1 - Cadastrar usuário")
    print("2 - Entrar")
    print("0 - Sair")
    return input("Escolha: ")

def principal():
    while True:
        opcao = exibir_menu()
        if opcao == "1":
            nome = input("Nome de usuário: ").strip()
            senha = input("Senha: ")
            if nome == "" or senha == "":
                print("Nome e senha são obrigatórios.")
                continue
            if ";" in nome or ";" in senha:
                print("O caractere ; não é permitido.")
                continue
            Usuario(nome, senha).cadastrar()
            print("Usuário cadastrado com sucesso!")
        elif opcao == "2":
            nome = input("Nome de usuário: ").strip()
            senha = input("Senha: ")
            if Usuario(nome, senha).autenticar():
                print(f"Bem-vindo(a) ao cofre digital, {nome}!")
            else:
                print("Usuário ou senha incorretos.")
        elif opcao == "0":
            print("Até logo!")
            break
        else:
            print("Opção inválida.")

principal()
```

```
=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 1
Nome de usuário: marina
Senha: 1234
```

```
Usuário cadastrado com sucesso!
```

Este é um ponto de partida. Sua dupla pode (e deve) acrescentar recursos, por exemplo: impedir nomes de usuário repetidos, guardar as reservas de cada cliente, listar as reservas após o login ou limitar as tentativas de senha.

19.4 Depuração coletiva

Troque de computador com outra dupla e **tente quebrar** o sistema: digite valores vazios, opções inexistentes, nomes repetidos. Registre cada bug encontrado:

O que foi feito	O que aconteceu	Como corrigir
Ex.: nome vazio	Usuário em branco foi cadastrado	Validar antes de cadastrar

> ⚠ **Atenção**
> Se o programa não encontrar `usuarios.txt` no VS Code, verifique se você abriu a **pasta** do projeto. No Colab, lembre que o arquivo some quando a sessão termina.

Aula 20 – Apresentação final, AV4 e encerramento

> **Nesta aula**
> **Objeto do conhecimento:** integração de todos os objetos do conhecimento do componente.
> **Habilidade a desenvolver:** desenvolver programas funcionais em Python, com capacidade de tomar decisões e preparo para desafios futuros.
> **Avaliação:** AV4 – avaliação somativa final.

20.1 Roteiro da apresentação

Cada dupla demonstrará o cofre digital funcionando, explicará as decisões de código e responderá a perguntas do professor e dos colegas.

1. **Demonstração:** cadastro, login e persistência (feche e abra o programa para provar que os dados permanecem).
2. **Explicação do código:** onde estão a classe, as funções, o laço do menu e a leitura/escrita do arquivo.
3. **Decisões:** por que escolheram essa organização? O que fariam diferente?
4. **Perguntas:** respondam em dupla e com tranquilidade; errar e corrigir também mostra aprendizado.

20.2 Critérios da AV4

Critério	O que será observado
Funcionamento	Cadastro, login e persistência funcionam sem travar.
Uso dos conteúdos	Variáveis, decisões, repetição, estruturas de dados, funções, arquivos e POO usados corretamente.
Organização do código	Nomes claros, comentários, funções e classe bem definidas.
Apresentação oral	Clareza ao explicar o que fez e por quê.
Colaboração	Participação equilibrada e trabalho em equipe.

Entrega da fase

Sistema final – cofre digital

Projeto completo da Agência Amazônia Tur Digital, apresentado em dupla.

20.3 E depois do curso?

Você concluiu a base da programação em Python. Os próximos passos naturais são:

- aprofundar a **Programação Orientada a Objetos** (herança, encapsulamento);
- aprender **tratamento de erros** (`try/except`) com mais profundidade;
- estudar **Pandas** e **análise de dados**, onde o Python se destaca;
- criar um portfólio no **GitHub** com os projetos do curso.

Dica

Programar se aprende praticando. Continue criando pequenos programas sobre assuntos que você gosta ou que fazem parte do seu trabalho.

Anexo A – Erros comuns e depuração

Todo programador encontra erros. Quando algo dá errado, o Python mostra o **tipo do erro**, a **linha** e uma descrição. Leia sempre a **última linha**: ela resume o problema.

Erro	Causa mais comum	Como resolver
`SyntaxError`	Parêntese, aspas ou `:` esquecidos.	Revise a linha indicada e a anterior.
`IndentationError`	Recuo incorreto.	Alinhe os blocos com 4 espaços.
`NameError`	Variável usada sem existir ou escrita diferente.	Confira a grafia; no Colab, execute as células de cima para baixo.
`TypeError`	Operação entre tipos incompatíveis (texto + número).	Converta com `int()`, `float()` ou `str()`.
`ValueError`	Valor inadequado, como `int("abc")`.	Valide a entrada do usuário.
`ZeroDivisionError`	Divisão por zero.	Teste o divisor antes de dividir.
`IndexError`	Posição inexistente na lista.	Verifique `len()` da lista.
`KeyError`	Chave inexistente no dicionário.	Confira a chave ou use `dicionario.get(chave)`.
`FileNotFoundError`	Arquivo não encontrado.	Confira o nome e a pasta de trabalho.

Tratando um erro com try/except

```
try:
    dias = int(input("Dias: "))
except ValueError:
    print("Digite apenas números inteiros!")
    dias = 1
```

Técnicas de depuração

- **Com print():** exiba valores intermediários, por exemplo `print("DEBUG:", total)`, para descobrir onde o programa foge do esperado.
- **Com o depurador do VS Code:** clique à esquerda do número da linha para criar um ponto de parada, pressione **F5** e avance linha a linha com **F10**, acompanhando as variáveis no painel lateral.
- **Teste com vários valores**, inclusive os inesperados (vazio, zero, negativo).

Anexo B – Resumo de comandos

Comando	Função
`print()`, `input()`	Saída e entrada de dados.
`int()`, `float()`, `str()`	Convertem tipos de dados.
`type()`, `len()`	Tipo de um valor; tamanho de texto ou coleção.
`if / elif / else`	Decisões.
`while`, `for`, `range()`	Repetições e sequências numéricas.
`break`, `continue`, `pass`	Controle de fluxo nos laços.
`lista.append()`, `.remove()`, `.sort()`	Operações com listas.
`dicionario.items()`, `.get()`	Percorrer e consultar dicionários.
`.upper()`, `.strip()`, `.split()`, `.replace()`	Manipulação de strings.
`def` e `return`	Criam funções e devolvem resultados.
`with open(...)`	Abre e fecha arquivos com segurança.
`class`, `__init__`, `self`	Classes, construtor e referência ao objeto.

Anexo C – Glossário

Termo	Significado
Algoritmo	Sequência finita de passos para resolver um problema.
Atributo	Dado que pertence a um objeto.
Célula	Bloco de um notebook que contém código ou texto.
Classe	Molde a partir do qual objetos são criados.
Depuração (debug)	Processo de encontrar e corrigir erros.
Dicionário	Coleção de pares chave-valor.
Função	Bloco de código nomeado e reutilizável.
Indentação	Recuo que define os blocos de código em Python.
Interpretador	Programa que lê e executa o código Python.
Método	Função que pertence a uma classe.
Notebook	Documento interativo (.ipynb) com código, texto e resultados.
Objeto	Exemplar criado a partir de uma classe.
Pseudocódigo	Descrição do algoritmo em linguagem simples, antes do código.
Script	Arquivo .py com instruções executadas em sequência.
String	Sequência de caracteres; texto.
Variável	Nome associado a um valor guardado na memória.

Anexo D – Gabarito das atividades

Existem várias maneiras corretas de resolver cada problema. Compare o **resultado** do seu programa com o esperado.

Aula 1

```
1. Leia o preço do passeio por pessoa
2. Leia o número de pessoas do grupo
3. Calcule: custo_total = preço × número de pessoas
4. Mostre o custo_total
```

- Atividades 2 e 3: a saída deve ser a mensagem digitada, por exemplo `Olá, Amazonas! Eu sou Marina.`

Aula 2

```
nome = input("Nome: ")
destino = input("Destino: ")
dias = int(input("Dias: "))
print(f"{nome} viajará para {destino} por {dias} dias.")
```

- (a) falta converter: `dias = int(input("Dias: ")) + 1`.
- (b) nome de variável não pode começar com número: `destino1 = "Parintins"`.
- (c) **TypeError** (texto + número): `print("Total: " + str(150))` ou `print(f"Total: {150}")`.

Aula 3

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
print(f"Média: {(n1 + n2 + n3) / 3:.2f}")

compra = float(input("Valor da compra: "))
pago = float(input("Valor pago: "))
print(f"Troco: R$ {pago - compra:.2f}")

print(1000 // 330)    # 3 dias inteiros
print(1000 % 330)     # sobram R$ 10
```

- `2 + 3 * 4 ** 2` = 2 + 3 × 16 = **50**.

Aula 4

- `tem_orcamento and tem_vaga` = `False`; `tem_orcamento or tem_vaga` = `True`; `not tem_vaga` = `True`.
- `1 <= dias <= 30` (ou `dias >= 1 and dias <= 30`).

Aula 5

```
idade = int(input("Idade: "))
```

```
if idade >= 18:
    print("Maior de idade")
else:
    print("Menor de idade")

nota = float(input("Nota: "))
if nota >= 7:
    print("Aprovado")
else:
    print("Reprovado")

idade = int(input("Idade: "))
estudante = input("Estudante? (s/n): ")
valor = 150.00
if idade >= 60 or estudante == "s":
    valor = valor * 0.8          # 20% de desconto
print(f"Valor: R$ {valor:.2f}")
```

Aula 6

```
media = float(input("Média: "))
if media >= 7:
    print("Aprovado")
elif media >= 5:
    print("Recuperação")
else:
    print("Reprovado")

if dias >= 5 and total <= 1500:
    print("Cliente ganha cortesia!")
```

Aula 8

```
nota = float(input("Nota (0 a 10): "))
while nota < 0 or nota > 10:
    print("Nota inválida!")
    nota = float(input("Nota (0 a 10): "))

soma = 0
gasto = float(input("Gasto (0 encerra): "))
while gasto != 0:
    soma += gasto
    gasto = float(input("Gasto (0 encerra): "))
print(f"Total: R$ {soma:.2f}")
```

Aula 9

```
n = int(input("Número: "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")

dias = int(input("Dias: "))
diaria = 100
soma = 0
for dia in range(1, dias + 1):
    soma += diaria
```

```
    diaria += 20
print(f"Soma das diárias: R$ {soma:.2f}")

for i in range(10, 0, -1):
    print(i)
print("Embarcar!")
```

Aula 10

- Saída: 1, 2 e 4. O 3 é pulado pelo `continue` e o laço termina no 5 pelo `break`, antes de imprimi-lo.

```
elif opcao == "3":
    print("Orçamento limite: R$ 1000,00")
```

Aula 11

```
destinos = ["Parintins", "Novo Airão", "Maués", "Presidente Figueiredo"]
print(destinos[0], destinos[-1])
destinos.append("Encontro das Águas")
destinos.sort()
print(destinos)

gastos = [990, 1250.5, 780]
print(max(gastos), min(gastos), sum(gastos) / len(gastos))   # 1250.5 780
1006.83...
```

Aula 12

```
passeio = {"nome": "Passeio de barco", "preco": 120.00, "duracao": 3}
for chave, valor in passeio.items():
    print(f"{chave}: {valor}")
```

- Lista: mutável, para conjuntos de itens que mudam (ex.: lista de clientes). Tupla: imutável, para dados fixos (ex.: coordenadas de uma cidade).

Aula 13

```
print("  joão PEDRO  ".strip().title())          # João Pedro

email = input("E-mail: ")
if "@" in email and "." in email:
    print("E-mail válido")
else:
    print("E-mail inválido")

codigo = f"{destino[:3].upper()}-{dias:02d}-{nome[:2].upper()}"
```

Aula 14

- `nomes[-1] = "Caio"`; `nomes[0:2] = ["Ana", "Bia"]`.
- `append(x)` adiciona `x` ao final; `remove(x)` retira a primeira ocorrência de `x`.
- `aluno["nota"] = 10` e `aluno["curso"] = "Python"`.
- `"MAUÉS"`.

Aula 15

```
def saudacao_cliente(nome):
    print(f"Olá, {nome}! Boa viagem!")

def calcular_total(dias, gasto_dia):
    return dias * gasto_dia

saudacao_cliente("Marina")
print(calcular_total(3, 330))
print(calcular_total(5, 200))
```

Aula 16

```
def eh_par(n):
    return n % 2 == 0

def buscar_destino(nome, catalogo):
    return catalogo.get(nome.strip().title())

print(buscar_destino("maués", catalogo))     # dados do destino
print(buscar_destino("Xingu", catalogo))     # None
```

- `print` apenas mostra o valor na tela; `return` devolve o valor ao programa, que pode guardá-lo e usá-lo.

Aula 17

```
destinos = ["Parintins", "Novo Airão", "Maués", "Presidente Figueiredo",
"Manaus"]
with open("destinos.txt", "w", encoding="utf-8") as f:
    for d in destinos:
        f.write(d + "\n")

with open("destinos.txt", "r", encoding="utf-8") as f:
    linhas = f.readlines()
for linha in linhas:
    print(linha.strip())
print(f"Total de linhas: {len(linhas)}")
```

- `"w"` apaga o conteúdo anterior e escreve do zero; `"a"` acrescenta ao final. Usar `"w"` por engano **apaga os dados** que já estavam no arquivo.

Aula 18

```
class Pacote:
    def __init__(self, destino, preco, dias):
        self.destino = destino
        self.preco = preco
        self.dias = dias

    def descricao(self):
        return f"{self.destino}: R$ {self.preco:.2f} em {self.dias} dias"

p1 = Pacote("Parintins", 950.00, 3)
p2 = Pacote("Maués", 1350.00, 4)
```

```
print(p1.descricao())
print(p2.descricao())

class Cliente:
    def __init__(self, nome, dias):
        self.nome = nome
        self.dias = dias

    def dobrar_dias(self):
        self.dias = self.dias * 2
```

Referências

- CETAM – CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS. **Curso de Qualificação Profissional em Linguagem de Programação Python: Plano Pedagógico de Curso (PPC) versão 001/DAC-2026**. Manaus, 2026.
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- DOWNEY, Allen B. Pense em Python: Pense como um cientista da computação. 3. ed. São Paulo: Novatec Editora, 2024.
- FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. Lógica de Programação: a construção de algoritmos e estruturas de dados com aplicações em Python. 4. ed. São Paulo: Pearson, 2022.
- GOOGLE. **Welcome to Colaboratory**. Disponível em: https://colab.research.google.com/. Acesso em: 2026.
- MENEZES, Nilo Ney Coutinho. Introdução à Programação com Python: algoritmos e lógica de programação para iniciantes. 4. ed. São Paulo: Novatec, 2024.
- MICROSOFT. **Python in Visual Studio Code**. Disponível em: https://code.visualstudio.com/docs/python/python-tutorial. Acesso em: 2026.
- OLIVEIRA, Cláudio Luís Vieira. Aprenda Lógica de Programação e Algoritmos com Implementações em Portugol, Scratch, C, Java, C# e Python. Rio de Janeiro: Ciência Moderna, 2016.
- PYTHON SOFTWARE FOUNDATION. **Documentação oficial do Python 3** (tutorial em português). Disponível em: https://docs.python.org/pt-br/3/tutorial/. Acesso em: 2026.
- PYTHON SOFTWARE FOUNDATION. **PEP 8 – Style Guide for Python Code**. Disponível em: https://peps.python.org/pep-0008/. Acesso em: 2026.
- PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. [S. l.]: Python Software Foundation, [s. d.]. Disponível em: https://docs.python.org/3/. Acesso em: 02 outubro de 2026.

- SWEIGART, Al. Automatize Tarefas Maçantes com Python. 3. ed. São Paulo: Novatec, 2015.